Naysha Kamboj

Rollno-2501730379

Section-F

Date-18-03-2026

Assignment 2

TASK 1

Feature	Arrays	Linked Lists
Memory layout	Contiguous	Non-contiguous (nodes linked by pointers)
Access time	O(1) random access	O(n) sequential traversal
Insertion/deletion	Costly at beginning/middle (O(n)); cheap at end if capacity available	Efficient at beginning (O(1)); middle/end requires traversal (O(n))
Memory overhead	Possible unused capacity; no extra pointers	Extra memory for pointers in each node
Real-world use cases	Fixed-size student marks, image pixel storage	Music playlist, dynamic memory allocation

Inside memory explanation:

- In an array, inserting at index 0 shifts all elements one step forward, which is costly.
- In a linked list, inserting at index 0 just creates a new node and points it to the old head — no shifting required.

Decision Scenarios

- (a) Marks of 60 students → Array (fixed size, direct index access).
- (b) Music playlist → Linked List (frequent middle insertions/deletions).
- (c) Browser Back history → Stack (LIFO).
- (d) Print queue → Queue (FIFO).

TASK 2

2.1

Formula: LOC(A[i]) = BA+i×w with BA = 2000, w = 4.

- A[0] = 2000 + 0×4 = 2000
- A[5] = 2000 + 5×4 = 2020
- A[17] = 2000 + 17×4 = 2068

2.2

Formula: LOC(B[i][j]) = BA+ i×C +j ×w with BA = 5000, C = 6, w = 2.

- B0][0] = 5000 + (0×6+0)×2 = 5000
- B2][3] = 5000 + (2×6+3)×2 = 5000 + 15×2 = 5030
- B3][5] = 5000 + (3×6+5)×2 = 5000 + 23×2 = 5046
- B4][0] → Out of range (only 0–3 valid for rows).

2.3

Concept Check-Arrays allow direct indexing because addresses are computed with a formula. Linked lists require traversal node by node, making access slower.

TASK 3

Key Methods

- append(x) → doubles capacity when full.
- pop() → remove last element.
- get(i), set(i,x) → O(1).
- insert(i,x), delete(i) → O(n) (shifting).
- size(), capacity() → track current usage.

Amortized Analysis:
Appending is usually O(1). Occasionally resizing costs O(n), but since it happens rarely, the average cost per append remains O(1).

Experiment Observation:
Capacity doubles when full. For N=5,10,20, you'll see size increasing linearly but capacity jumps (e.g., 4 → 8 → 16). Pattern: capacity grows exponentially, size grows linearly.

TASK 4

Singly Linked List

- insert_at_beginning(x) → O(1)

- insert_at_end(x) → O(n) unless tail pointer maintained
- delete_by_value(x) → O(n)
- search(x) → O(n)
- traverse() → O(n)

Doubly Linked List

- insert_after_node(key,x) → O(n)
- delete_at_position(pos) → O(n), but easier than singly since backtracking is possible.

Circular Linked List

Last node points back to head. Useful in round-robin scheduling or looping playlists.

Task 5

5.1

Stack (LIFO)

- Implement with linked list head as **top** (push/pop O(1)).
- Operations: push, pop, peek, is_empty.

5.2

Queue (FIFO)

- Maintain **head** (dequeue) and **tail** (enqueue).
- Operations: enqueue, dequeue, front, is_empty.

Operation	Array	Linked Lists
Access	O(1)	O(n)
Insert at beginning	O(n)	O(1)
Insert at end	O(1) if capacity	O(1) with tail
Delete at beginning	O(n)	O(1)
Delete at end	O(1) if capacity	O(n) unless tail

TASK 6

Logic

- Push opening brackets onto stack.
- Pop when a closing bracket matches top.
- Invalid if mismatch or stack empty at wrong time.
- Balanced if stack empty at end.

Example Stack States

- "([])" → push (, push [, pop], pop), stack empty → balanced.
- "([]" → push (, push [, pop], stack has (left → not balanced.

OUTPUTS

```
PS C:\Users\Naysha\OneDrive\Desktop\DSA>  & 'c:\python313\python.exe' 'c:\Users\Naysha\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64064' '--' 'C:\Users\Naysha\OneDrive\Desktop\DSA\unit2_demo_tests.py'
=== T1. DynamicArray ===
Appended 1: size=1, capacity=2
Appended 2: size=2, capacity=2
Appended 3: size=3, capacity=4
Appended 4: size=4, capacity=4
Appended 5: size=5, capacity=8
Array: [1, 2, 3, 4, 5]
After 2 pops: [1, 2, 3]
After insert/delete: [1, 99, 3]

=== T2. Singly Linked List ===
Traverse: [1, 2, 3, 4, 5]
After deleting 3: [1, 2, 4, 5]
Search 4: True
Search 100: False

=== T3. Stack ===
Peek: 30
Pop: 30
Pop: 20
Peek after pops: 10

=== T4. Queue ===
Front: 7
Dequeue: 7
Dequeue: 8
Front after dequeues: 9

=== T5. Parentheses Checker ===
([]): True
([)]: False
(((: False
Empty string: True
PS C:\Users\Naysha\OneDrive\Desktop\DSA>
```